

# Trigger de Abate Automático de Estoque

Automação transacional no PostgreSQL garantindo consistência entre Pagamentos e Produtos

Para garantir a integridade dos dados e evitar concorrência (vender peças sem estoque), a melhor abordagem é delegar o abate de estoque diretamente ao motor do banco de dados PostgreSQL por meio de uma **Function** e uma **Trigger** acionada na mudança de status do pagamento.

## 1. A Função da Trigger (PL/pgSQL)

Esta função é executada sempre que a tabela pagamentos sofre uma atualização. Ela verifica se o status transicionou para 'APROVADO' e percorre todos os itens do pedido associado reduzindo o estoque de forma atômica.

```
CREATE OR REPLACE FUNCTION fn_abater_estoque_apos_pagamento()
RETURNS TRIGGER AS $$
DECLARE
    item RECORD;
BEGIN
    -- Executa apenas quando o status muda para 'APROVADO' (evita execução duplicada)
    IF (NEW.status = 'APROVADO' AND (OLD.status IS NULL OR OLD.status <> 'APROVADO')) THEN

        -- Percorre todos os produtos vinculados ao pedido pago
        FOR item IN
            SELECT produto_id, quantidade
            FROM pedido_itens
            WHERE pedido_id = NEW.pedido_id
        LOOP
            -- Tenta atualizar o estoque garantindo que haja quantidade disponível
            UPDATE produtos
            SET quantidade_estoque = quantidade_estoque - item.quantidade
            WHERE id = item.produto_id
            AND quantidade_estoque >= item.quantidade;

            -- Se nenhuma linha for afetada, o estoque era insuficiente
            IF NOT FOUND THEN
                RAISE EXCEPTION 'Estoque insuficiente para o produto ID % no Pedido %',
                    item.produto_id, NEW.pedido_id;
            END IF;
        END LOOP;

        -- Atualiza também o status do pedido para 'EM_PREPARACAO'
        UPDATE pedidos
        SET status = 'EM_PREPARACAO', atualizado_em = NOW()
        WHERE id = NEW.pedido_id;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

## 2. A Criação da Trigger

A trigger liga a função criada acima aos eventos da tabela pagamentos. Ela é executada AFTER UPDATE para cada linha alterada.

```
DROP TRIGGER IF EXISTS trg_abater_estoque ON pagamentos;

CREATE TRIGGER trg_abater_estoque
AFTER UPDATE ON pagamentos
FOR EACH ROW
EXECUTE FUNCTION fn_abater_estoque_apos_pagamento();
```

### ⚠ Segurança contra Race Conditions e Rollback Total

Se qualquer produto do pedido não tiver estoque suficiente no exato momento da aprovação do pagamento, o PostgreSQL dispara um RAISE EXCEPTION. Isso aborta a transação inteira e faz o **rollback automático** — o status do pagamento não muda para APROVADO no banco de dados e nenhum produto tem seu estoque alterado.

## 3. Cenário de Teste / Validação

Você pode testar o comportamento da automação executando a seguinte simulação no seu cliente SQL:

```
-- Simula o webhook do Mercado Pago atualizando o pagamento para APROVADO
UPDATE pagamentos
SET status = 'APROVADO', pago_em = NOW()
WHERE pedido_id = 1024;
```

## Vantagens Arquiteturais

- **Consistência ACID:** A lógica é transacional. Não há risco da sua aplicação Python falhar no meio do processo e deixar o estoque inconsistente.
- **Desacoplamento:** O microserviço de Webhooks/Pagamentos só precisa atualizar uma linha no banco de dados. O controle de estoque acontece de forma transparente no SGBD.
- **Sincronização com Loja Física:** Se o sistema PDV da loja física estiver conectado ao mesmo PostgreSQL, a peça fica indisponível para venda presencial no milissegundo em que o Pix é aprovado.